HUDM 5001 Programming for Data Science
Fall 2026 Teachers College, Columbia University

Midterm Exam

Tuesday, October 27, 2026
This exam covers everything through Week 7, including control structures and iterables.

Name: _____ **UNI:** _____

Time limit: 100 minutes.

Rules

1. Closed book. You may use **one** one-sided, letter-size sheet of notes (your "cheat sheet") and a calculator.
2. No computers, phones, tablets, smartwatches, earbuds, or AI tools. Put them away before the exam starts.
3. You will **not** write any code. Every question asks you to read, trace, explain, or choose.
4. Part A: choose one answer for each question and mark it on the answer grid on page 2. Only the grid is scored.
5. Part B: write your answers in the space provided. You may cite line numbers (for example, "line 12") instead of copying code. Show your reasoning, because partial credit is given.
6. Unless a question says otherwise, assume that `import json, os, sqlite3`, `import numpy as np` and `import pandas as pd` have already been run, that each snippet runs by itself in a fresh Python 3 session, and that no warning messages appear.

Part	Points	Score
A. Multiple choice (20 questions × 2.5 points)	50	
B1. Reading comprehension: academic research	25	
Total	**75**	

Do not turn the page until you are told to begin.

Designed by Chenguang Pan (cp3280@tc.columbia.edu), refined by Claude.ai.

Part A: Answer grid

Mark **exactly one** letter for each question. Only this grid is scored for Part A. If you change an answer, erase the old mark completely.

1.	A	B	C	D	**11.**	A	B	C	D
2.	A	B	C	D	**12.**	A	B	C	D
3.	A	B	C	D	**13.**	A	B	C	D
4.	A	B	C	D	**14.**	A	B	C	D
5.	A	B	C	D	**15.**	A	B	C	D
6.	A	B	C	D	**16.**	A	B	C	D
7.	A	B	C	D	**17.**	A	B	C	D
8.	A	B	C	D	**18.**	A	B	C	D
9.	A	B	C	D	**19.**	A	B	C	D
10.	A	B	C	D	**20.**	A	B	C	D

Part A: Multiple choice

20 questions × 2.5 points = 50 points. Choose the single best answer for each question and mark it on the answer grid (page 2).

1. What does this code print?

```
course = "HUDM"
code = course
course = course + " 5001"
print(code, "and", course)
```

A. `HUDM 5001 and HUDM 5001`
B. `HUDM and HUDM 5001`
C. `HUDM 5001 and HUDM`
D. `HUDM and HUDM`

2. The Series `sizes` is built below. Which expression returns 25 **by its label**?

```
sizes = pd.Series([12, 30, 25, 8],
                  index=["a", "b", "c", "d"])
```

A. `sizes.iloc["c"]`
B. `sizes.loc[2]`
C. `sizes.iloc[3]`
D. `sizes.loc["c"]`

3. What does this code print?

```
df = pd.DataFrame({"id":    [1, 2, 3, 4],
                   "score": [55, 82, 91, 67],
                   "grade": ["F", "B", "A", "D"]})
print(df.shape, df["score"].shape, df[df["score"] > 60].shape)
```

Tip. Selecting one column with single brackets, `df["col"]`, gives a pandas Series, not a DataFrame.

A. `(4, 3) (4,) (3, 3)`
B. `(3, 4) (4,) (3, 3)`
C. `(4, 3) (4, 1) (3, 3)`
D. `(4, 3) (4,) (3, 1)`

4. What does this code print?

```
df = pd.DataFrame({"name":  ["Ana", "Ben", "Cai", "Dee", "Eli"],
                   "score": [72, 65, 88, 59, 91]})
conn = sqlite3.connect("class.db")
df.to_sql("roster", conn, index=False, if_exists="replace")
query = "SELECT COUNT(*) AS n FROM roster WHERE score > 70"
res = pd.read_sql_query(query, conn)
conn.close()
print(res.shape, res.loc[0, "n"])
```

A. `(5, 2) 3`
B. `(1, 1) 5`
C. `(1, 1) 3`
D. `(3, 1) 3`

5. What does this code print?

```
print(17 // 5, 17 % 5, -7 // 2)
```

A. `3 2 -3`
B. `3 2 -4`
C. `3.4 2 -3.5`
D. `2 3 -4`

6. Each line below is run by itself. Which one raises a `ValueError`?

A. `int("38")`
B. `int(3.8)`
C. `int("3.8")`
D. `float("3.8")`

7. What does this code print?

```
lst = [1, 2, 3]
arr = np.array([1, 2, 3])
print(lst * 2, arr * 2)
```

A. `[1, 2, 3, 1, 2, 3] [2 4 6]`
B. `[2, 4, 6] [2 4 6]`
C. `[1, 2, 3, 1, 2, 3] [1 2 3 1 2 3]`
D. `[2, 4, 6] [1 2 3 1 2 3]`

8. What does this code print?

```
s = pd.Series([4, np.nan, 10, np.nan])
print(len(s), s.count(), s.mean())
```

Tip. `.count()` counts only the values that are not missing. It does not count NaN.

A. `4 4 3.5`
B. `2 2 7.0`
C. `4 2 3.5`
D. `4 2 7.0`

9. What does this code print?

```
votes = pd.Series(["yes", "no", "yes", "yes", "no"])
counts = votes.value_counts()
shares = votes.value_counts(normalize=True)
print(counts["yes"], shares["no"])
```

A. `3 2`
B. `3 0.4`
C. `0.6 0.4`
D. `3 40.0`

10. What does this code print?

```
nums = [5, 2, 9]
a = sorted(nums)
b = nums.sort()
print(a, b, nums)
```

Tip. `a_list.sort()` sorts the list itself and returns None.

A. `[2, 5, 9] [2, 5, 9] [5, 2, 9]`
B. `[2, 5, 9] [2, 5, 9] [2, 5, 9]`
C. `[2, 5, 9] None [2, 5, 9]`
D. `[2, 5, 9] None [5, 2, 9]`

11. What does this code print?

```
df = pd.DataFrame({"id": [1, 1, 2, 3],
                   "x":  [5, 5, 6, 7]})
df.drop_duplicates()
print(df.shape[0], df.drop_duplicates().shape[0])
```

Tip. `.drop_duplicates()` returns a new table. It does not change `df` itself.

A. `3 3`
B. `4 3`
C. `4 4`
D. `3 4`

12. What does this code print?

```
n = 100
steps = 0
while n > 1:
    n = n // 3
    steps += 1
print(n, steps)
```

Tip. `//` rounds down, so `2 // 3` is 0.

A. `0 5`
B. `3 3`
C. `1 3`
D. `1 4`

13. What does this code print?

```
df = pd.DataFrame({"q1": [4, 2, np.nan],
                   "q2": [5, np.nan, 3]})
df["total"] = df["q1"] + df["q2"]
print(df["total"].sum(), df["total"].count())
```

Tip. A number plus a missing value is missing: 4 + NaN gives NaN.

A. `14.0 3`
B. `9.0 3`
C. `9.0 1`
D. `nan 1`

14. What does this code print?

```
values = [4, -1, 6, 0, 9, 3]
total = 0
for v in values:
    if v < 0:
        continue
    if v == 0:
        break
    total += v
print(total)
```

A. `10`
B. `22`
C. `9`
D. `21`

15. What does this code print?

```
names  = ["Ana", "Ben", "Cai", "Dee"]
scores = [72, 58, 91, 65]
flags = []
for name, s in zip(names, scores):
    flags.append(name if s < 65 else "ok")
print(flags)
```

A. `['Ana', 'ok', 'Cai', 'Dee']`
B. `['ok', 'Ben', 'ok', 'ok']`
C. `['ok', 'Ben', 'ok', 'Dee']`
D. `['ok', 58, 'ok', 'ok']`

16. What does this code print?

```
a = pd.DataFrame({"score": [70, 80]})
b = pd.DataFrame({"score": [90, 60, 75]})
both = pd.concat([a, b])
print(both.shape, len(both.loc[0]), both.iloc[2]["score"])
```

Tip. Unless you set `ignore_index=True`, `pd.concat` keeps each table's original row labels (its index).

A. `(5, 1) 1 90`
B. `(5, 1) 2 75`
C. `(5, 1) 2 90`
D. `(3, 2) 1 90`

17. A SQLite table `grades` holds exactly the rows of `df` below. Which pandas expression gives the same names, in the same order, as `query`?

```
df = pd.DataFrame({"name":  ["Ann", "Ben", "Cy", "Dee", "Eve"],
                   "score": [85, 92, 78, 95, 60]})
query = """SELECT name FROM grades
           WHERE score >= 80
           ORDER BY score DESC"""
```

A. `list(df[df["score"] > 80].sort_values("score")["name"])`
B. `list(df.sort_values("score", ascending=False)["name"])`
C. `list(df[df["score"] >= 80]["name"])`
D. `list(df[df["score"] >= 80].sort_values("score", ascending=False)["name"])`

18. A SQLite table `orders` is shown below, and the cursor `cur` is connected to its database. What does the code print?

```
region | amount
-------+-------
east   | 40
west   | 70
east   | 60
north  | 30
west   | 45
north  | 20
```

```
query = """SELECT region, SUM(amount) FROM orders
           GROUP BY region ORDER BY region"""
rows = cur.execute(query).fetchall()
big = 0
for region, total in rows:
    if total >= 100:
        big += 1
print(len(rows), big, rows[0])
```

A. `3 2 ('east', 100)`
B. `6 2 ('east', 40)`
C. `3 1 ('east', 100)`
D. `3 2 ('west', 115)`

19. What does this code print?

```
data = {"math": [80, 95, 70], "art": [70], "bio": [60, 90, 75]}
with open("marks.json", "w") as f:
    json.dump(data, f)
with open("marks.json", "r") as f:
    back = json.load(f)
best_subject = ""
best_n = 0
for subject, marks in back.items():
    if len(marks) > best_n:
        best_n = len(marks)
        best_subject = subject
print(best_subject, best_n, back == data)
```

A. `bio 3 True`
B. `math 3 False`
C. `art 1 True`
D. `math 3 True`

20. What does this code print?

```
left = pd.DataFrame({"id":   [1, 2, 3, 4],
                     "math": [70, None, 85, 90]})
right = pd.DataFrame({"id":      [2, 3, 5],
                      "reading": [88, None, 75]})
m = pd.merge(left, right, on="id", how="outer")
report = {}
for col in m.columns:
    n = int(m[col].isna().sum())
    if n > 0:
        report[col] = n
print(report)
```

A. `{'math': 2, 'reading': 3}`
B. `{'math': 1, 'reading': 1}`
C. `{'math': 1, 'reading': 3}`
D. `{'id': 0, 'math': 2, 'reading': 3}`

Part B: Reading comprehension

One script, 25 points. Read the background, study the data and the script, then answer the questions that follow. Each question says which version of the code it is about. You never need to write code. Describe fixes in words and cite line numbers.

B1. [25 pts] Academic research: a math-anxiety follow-up

Setting. Jordan is a research assistant on a small study of a six-week mindfulness program that aims to lower math anxiety in ninth graders. Ten students filled out a math-anxiety questionnaire before the program started. Scores run from 20 to 60, and a higher score means more anxious. Five students were in the program (treatment) and five were in a comparison class (control). Two of them, one in each group, moved away before the post-test. Jordan wrote the script `anxiety_followup.py` (next page) to compare the two groups. The principal investigator (PI) gave Jordan these rules:

R1. Analyze only students who took both the pre-test and the post-test.

R2. A student's change is the pre-test score minus the post-test score. Anxiety going down is good, so a positive change means improvement.

R3. A change of 5 or more (≥ 5) counts as "improved", a change of -5 or lower (≤ -5) as "worse", and anything in between as "stable".

R4. For each group, report the number of students and the percent labeled "improved", rounded to 1 decimal.

R5. Report the gap in percent improved (treatment minus control), and save everything to `summary.json`.

The data. The two input files, exactly as pandas displays them after `pd.read_csv`:

`pre.csv`

	student_id	group	anxiety_pre
0	101	treatment	42
1	102	treatment	38
2	103	treatment	30
3	104	treatment	45
4	105	treatment	50
5	201	control	36
6	202	control	31
7	203	control	44
8	204	control	40
9	205	control	33

`post.csv`

	student_id	anxiety_post
0	101	34
1	102	33
2	103	30
3	105	38
4	201	34
5	202	36
6	203	38
7	205	32

Jordan's script: `anxiety_followup.py`

```python
# anxiety_followup.py: compare the change in math anxiety between two groups
import json
import pandas as pd

pre = pd.read_csv("pre.csv")      # student_id, group, anxiety_pre
post = pd.read_csv("post.csv")    # student_id, anxiety_post
print("pre:", pre.shape[0], "| post:", post.shape[0])

# Step 1: put each student's two scores side by side
df = pd.merge(pre, post, on="student_id", how="left")
print("rows after merge:", df.shape[0])

# Step 2: change = pre minus post (a positive change means less anxious)
df["change"] = df["anxiety_pre"] - df["anxiety_post"]

# Step 3: give every student a label
labels = []
for c in df["change"]:
    if c >= 5:
        labels.append("improved")
    elif c > -5:
        labels.append("stable")
    else:
        labels.append("worse")
df["label"] = labels

# Step 4: one summary line per group
summary = {}
for group in ["control", "Treatment"]:
    sub = df[df["group"] == group]
    n = sub.shape[0]
    n_improved = int((sub["label"] == "improved").sum())
    pct = round(n_improved / n * 100, 1)
    summary[group] = {"n": n, "improved": n_improved, "pct": pct}
    print(group, n, pct)

# Step 5: the gap between the groups, then save everything for the PI
gap = summary["treatment"]["pct"] - summary["control"]["pct"]
print("gap in % improved:", gap)
summary["gap"] = gap
with open("summary.json", "w") as f:
    json.dump(summary, f)
print("saved summary.json")
```

Terminal output. This is what the terminal shows when Jordan runs the original script:

```
$ python anxiety_followup.py
pre: 10 | post: 8
rows after merge: 10
control 5 20.0
Traceback (most recent call last):
  File "anxiety_followup.py", line 33, in <module>
    pct = round(n_improved / n * 100, 1)
                ~~~~~~~~~~~^~~
ZeroDivisionError: division by zero
```

About the script. It contains exactly one bug and one PI's rule violation. The bug stops the script with an error message. The rule violation does not stop anything, but it makes the numbers wrong. Each question below says which version of the code it is about.

(a) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Line 42 saves the dictionary `summary` to the file `summary.json`. Explain what the file contains: its keys, and what each value holds. You do not need to calculate any numbers.

(b) **[4 pts]** *Code version: Original script (this answer does not depend on the bug or the rule violation).* In plain English, what does Step 3 (lines 16–25) do? Why does the order of the conditions on lines 19 and 21 matter?

(c) **[6 pts]** *Code version: Original script.* Look at the terminal output above: the script prints three lines and then stops with an error. (i) Which line raises the error, and what does the message mean? (ii) Which other line is the real cause? Use the data and the printed lines as evidence. (iii) Describe in words how to fix it.

(d) **[6 pts]** *Code version: Assume the error in (c) has been fixed, so the script runs to the end.* Even so, one line breaks one of the PI's rules, and Python does not give any error message. (i) Which line is it, and which rule does it break? (ii) Explain the mistake, including what happens to students 104 and 204. (iii) Explain how to revise that line.

(e) **[5 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Rule R3 defines the three labels. Does Step 3 follow R3, including the boundary values 5 and -5? Answer Yes or No, then explain. Use students 102 and 202 in your answer: their changes are exactly 5 and -5.

Need more room? Continue any Part B answer in the space below, and write the letter of the question you are continuing.

End of exam